Computação Distribuída

SCTEC – Sistema de Controle de Telescópio Espacial Compartilhado

Entregas 1, 2 e 3

Aluno: Pedro Roberto Fernandes Noronha

RA: 10443434

Professor: Mario

Repositório: https://github.com/peedroroberto12/Sistema-de-Controle-de-Telesc-pio-Espacial-Compartilhado-SCTEC-

Sobre o projeto

O objetivo do projeto é criar o sistema de agendamento de um telescópio espacial que vai ser disputado por cientistas do mundo todo. Cada horário de observação (slot de 5 minutos) só pode ter um dono, então o sistema precisa ser justo e não pode deixar duas pessoas reservarem o mesmo horário.

O sistema foi dividido em dois serviços:

- Serviço de Agendamento (Python/Flask): cuida das regras de negócio e do banco de dados. É o "cérebro" do sistema.
- Serviço Coordenador (Node.js/Express): controla quem pode mexer em cada horário, usando lock. É o "porteiro".

Entrega 1 – Blueprint da API

Nessa etapa não teve código, só a especificação do sistema. Foram criados três arquivos:

- MODELOS.md: as entidades do sistema (Cientista, Telescópio e Agendamento), seus campos e relacionamentos.
- API.md: todos os endpoints dos dois serviços, com exemplos de requisição, resposta e os links HATEOAS.
- LOGGING.md: o formato dos logs de aplicação (texto) e dos logs de auditoria (JSON).

Sobre o HATEOAS: as respostas da API trazem um campo _links com as próximas ações possíveis. Por exemplo, um agendamento ativo vem com o link de cancelar, e um já cancelado não vem. Assim o cliente não precisa adivinhar o que pode fazer.

Sobre os logs: separamos o log de aplicação, que serve pra depurar o código, do log de auditoria, que é o registro oficial de quem fez o quê e quando. É o log de auditoria que resolve uma disputa entre cientistas.

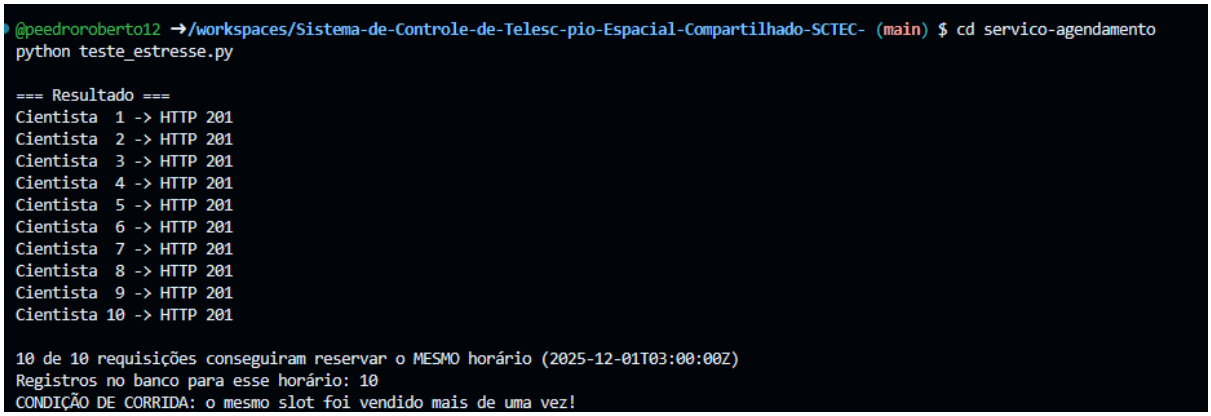
Entrega 2 – Sistema inicial e prova da condição de corrida

O que foi feito

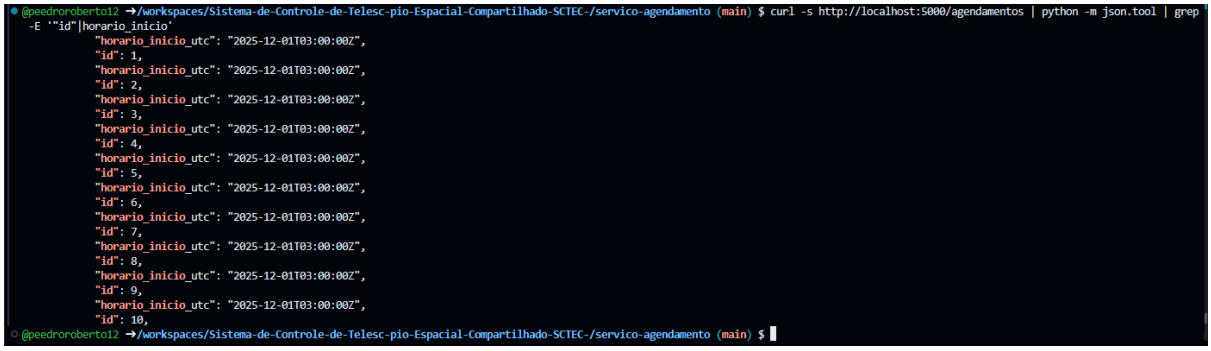
Implementei o Serviço de Agendamento em Flask com SQLAlchemy e SQLite, com a rota POST /agendamentos e os links HATEOAS. O logging grava no arquivo app.log e no terminal: os logs de aplicação ficam no nível INFO e os de auditoria usam um nível customizado AUDIT, em JSON.

Pra testar, criei o script teste_estresse.py, que dispara 10 requisições ao mesmo tempo tentando reservar o mesmo horário (2025-12-01T03:00:00Z). Usei uma barreira (threading.Barrier) pra garantir que todas saíssem juntas.

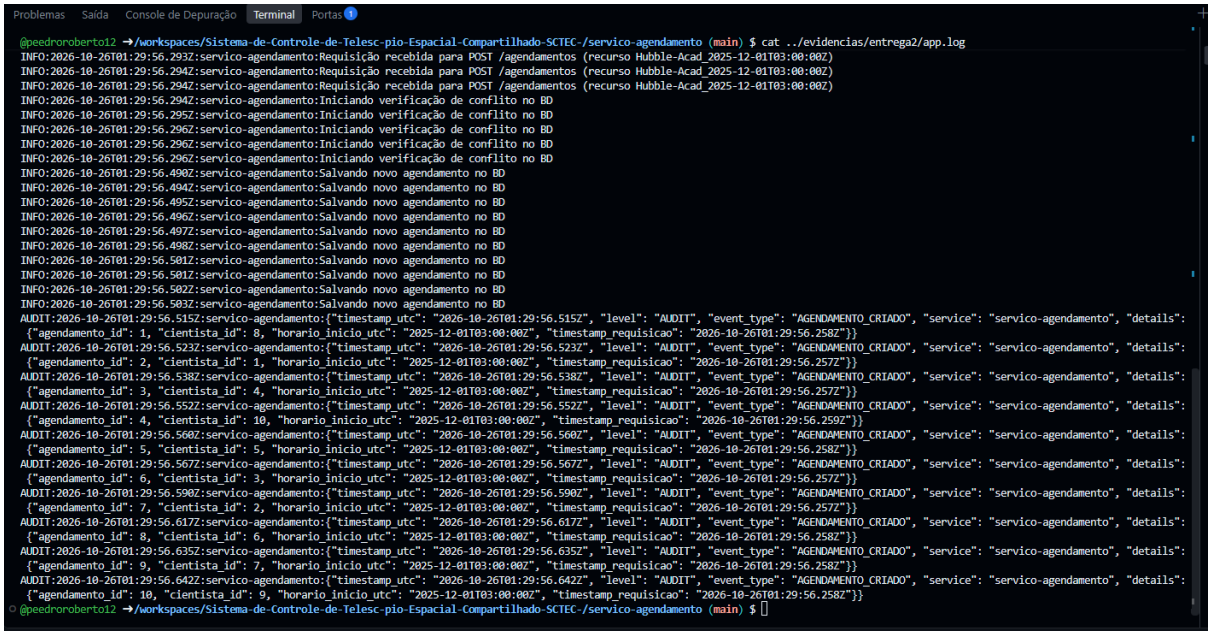
Resultado



As 10 requisições receberam 201 Created, ou seja, todas conseguiram reservar o mesmo horário.



No banco ficaram 10 registros pro mesmo slot, o que quebra a regra principal do sistema.



Análise

O log mostra exatamente onde está o problema. As 10 linhas "Iniciando verificação de conflito no BD" aparecem antes da primeira linha "Salvando novo agendamento no BD". Isso quer dizer que todas as requisições consultaram o banco quando ele ainda estava vazio, todas concluíram que o horário estava livre e todas salvaram. No final aparecem 10 logs de auditoria AGENDAMENTO_CRIADO pro mesmo horário.

Isso é uma condição de corrida. Verificar e salvar são dois passos separados, e nada impede outra requisição de entrar no meio. É como várias pessoas olhando a mesma cadeira vazia no cinema e sentando ao mesmo tempo. Esse trecho do código é a seção crítica e precisa de exclusão mútua, que foi resolvida na Entrega 3.

Entrega 3 – O Coordenador

O que foi feito

Criei o Serviço Coordenador em Node.js com Express. Ele tem os endpoints POST /lock e POST /unlock e guarda em memória quais recursos estão travados. O recurso é identificado pelo telescópio mais o horário, por exemplo Hubble-Acad_2025-12-01T03:00:00Z.

No Flask, a rota de agendamento passou a pedir o lock antes de mexer no banco:

- Se o coordenador responde 200, o Flask verifica o banco, salva e gera o log de auditoria.
- Se responde 409, o horário já está sendo usado e o Flask devolve 409 pro cliente sem tocar no banco.
- O unlock fica dentro de um bloco finally, então o lock é liberado mesmo se acontecer algum erro.

Também coloquei um tempo de expiração (TTL) de 10 segundos no lock. Se o Flask cair segurando um lock, ele não fica travado pra sempre.

Resultado

```
   0c692b9..bb5e473  main -> main
@peedroroberto12 ➜/workspaces/Sistema-de-Controle-de-Telesc-pio-Espacial-Compartilhado-SCTEC- (main) $ cd /workspaces/Sistema*/servico-agendamento
python teste_estresse.py

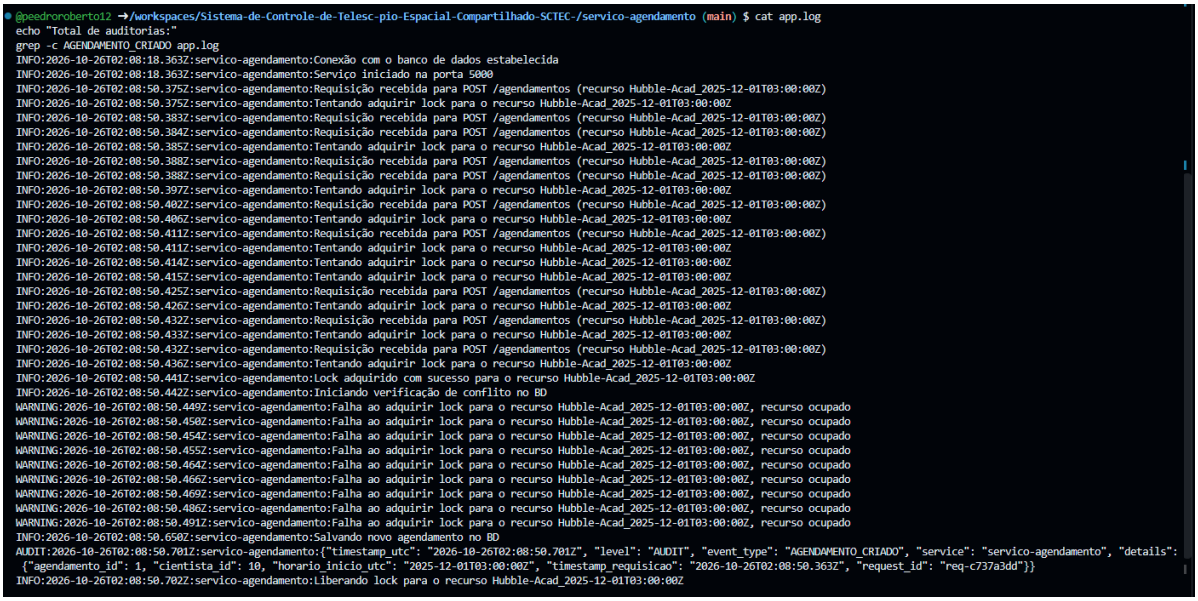
=== Resultado ===
Cientista  1 -> HTTP 409
Cientista  2 -> HTTP 409
Cientista  3 -> HTTP 409
Cientista  4 -> HTTP 409
Cientista  5 -> HTTP 201
Cientista  6 -> HTTP 409
Cientista  7 -> HTTP 409
Cientista  8 -> HTTP 409
Cientista  9 -> HTTP 409
Cientista 10 -> HTTP 409

1 de 10 requisições conseguiram reservar o MESMO horário (2025-12-01T03:00:00Z)
Registros no banco para esse horário: 1
@peedroroberto12 ➜/workspaces/Sistema-de-Controle-de-Telesc-pio-Espacial-Compartilhado-SCTEC-/servico-agendamento (main) $
```

Agora só uma requisição conseguiu reservar e as outras nove receberam 409 Conflict. O banco ficou com apenas um registro.

```
@peedroroberto12 ➜/workspaces/Sistema-de-Controle-de-Telesc-pio-Espacial-Compartilhado-SCTEC-/evidencias/entrega2 (main) $ cd /workspaces/Sistema*/servico-coordenador
node server.js
[2026-10-26T02:05:55.071Z] [servico-coordenador] Serviço iniciado na porta 3000

[2026-10-26T02:08:50.426Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-c737a3dd)
[2026-10-26T02:08:50.430Z] [servico-coordenador] Lock concedido para o recurso Hubble-Acad_2025-12-01T03:00:00Z
[2026-10-26T02:08:50.442Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-5860367d)
[2026-10-26T02:08:50.443Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.446Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-7e0db7ec)
[2026-10-26T02:08:50.446Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.448Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-00303c48)
[2026-10-26T02:08:50.448Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.451Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-5569207d)
[2026-10-26T02:08:50.452Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.457Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-fd173932)
[2026-10-26T02:08:50.460Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.462Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-3ed7116f)
[2026-10-26T02:08:50.464Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.465Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-c396bf90)
[2026-10-26T02:08:50.467Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.472Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-679d5b79)
[2026-10-26T02:08:50.478Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.486Z] [servico-coordenador] Recebido pedido de lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z (dono req-91764761)
[2026-10-26T02:08:50.486Z] [servico-coordenador] Recurso Hubble-Acad_2025-12-01T03:00:00Z já em uso, negando lock
[2026-10-26T02:08:50.706Z] [servico-coordenador] Recebido pedido de unlock para o recurso Hubble-Acad_2025-12-01T03:00:00Z
[2026-10-26T02:08:50.706Z] [servico-coordenador] Lock para o recurso Hubble-Acad_2025-12-01T03:00:00Z liberado
```



Análise

Os logs dos dois serviços contam a mesma história. O Node mostra um "Lock concedido" e nove "já em uso, negando lock". O Flask mostra um "Lock adquirido com sucesso", nove "Falha ao adquirir lock" e só um log de auditoria AGENDAMENTO_CRIADO. Depois que a primeira requisição termina, aparece o "Lock liberado" no Node.

O mesmo time.sleep da Entrega 2 continua no código, mas agora não causa problema, porque enquanto uma requisição segura o lock nenhuma outra entra na seção crítica. O coordenador transformou acessos simultâneos em acessos um de cada vez.

E por que o próprio Node não tem condição de corrida? O JavaScript roda numa thread só, e as linhas que verificam se o recurso está livre e travam ele são executadas direto, sem nenhuma pausa no meio. Então nenhuma outra requisição consegue entrar entre esses dois passos. Já o Flask atende várias requisições ao mesmo tempo, e era aí que elas se atropelavam.

Comparando os dois logs, o request_id req-c737a3dd da auditoria é o mesmo dono que recebeu o "Lock concedido" no coordenador. Isso prova que a requisição que ganhou o lock foi a mesma que salvou o agendamento. Também dá pra ver que o lock ficou travado por cerca de 0,28 s (de 50.430 a 50.706), e todas as requisições que chegaram nesse intervalo foram barradas. O vencedor muda a cada execução, porque vence quem chega primeiro no coordenador: no teste do Print 4 ganhou o cientista 5, e na execução dos Prints 5 e 6 ganhou o cientista 10. Em todas, só um conseguiu reservar.

Conclusão

Na Entrega 2 deu pra ver na prática que um sistema que atende várias requisições ao mesmo tempo pode quebrar as próprias regras, mesmo com o código "certo". Os logs foram essenciais pra provar e entender o problema. Na Entrega 3, um coordenador centralizado resolveu a disputa garantindo exclusão mútua, e o sistema passou a ter exatamente um vencedor por horário.